

25. Multiple Beans of the Same Type

A common problem occurs when Spring finds **more than one Bean of the same type**.

For example, suppose we have:

```
Payment
├── CardPayment
└── UpiPayment
```

Both classes implement the same interface:

```
public interface Payment {
    void pay();
}
```

Card Payment

```
@Component
public class CardPayment implements Payment {

    @Override
    public void pay() {
        System.out.println("Paying via Card");
    }
}
```

UPI Payment

```
@Component
public class UpiPayment implements Payment {

    @Override
    public void pay() {
        System.out.println("Paying via UPI");
    }
}
```

Now suppose another Bean requires:

```
Payment payment;
```

Spring finds:

```
Payment
  ↑
  ├── CardPayment Bean
  └── UpiPayment Bean
```

Spring now has a problem:

Which Bean should be injected?

This creates an ambiguity.

26. The Ambiguity Problem

Consider:

```
@Component
public class OrderService {

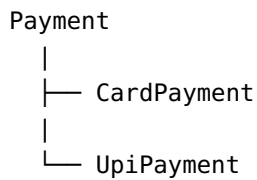
    private Payment payment;

    public OrderService(Payment payment) {
        this.payment = payment;
    }
}
```

Spring tries to create:

```
OrderService
  |
  ↓
Payment
```

But there are two possible Beans:



Spring cannot automatically decide which implementation should be injected.

Therefore, we need to tell Spring which Bean should be preferred.

There are two important solutions:

1. `@Primary`
2. `@Qualifier`

27. `@Primary`

What is `@Primary`?

`@Primary` tells Spring:

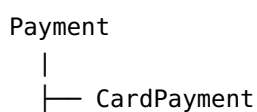
If multiple Beans of the same type are available, prefer this Bean by default.

Example:

```
@Component
@Primary
public class UpiPayment implements Payment {

    @Override
    public void pay() {
        System.out.println("Paying via UPI");
    }
}
```

Now:



```
|  
└─ UpiPayment ← @Primary
```

If Spring needs a `Payment` Bean and there is no more specific instruction, it selects:

UpiPayment

28. Why Do We Need `@Primary`?

Suppose:

```
@Component  
public class CardPayment implements Payment {  
}
```

and:

```
@Component  
@Primary  
public class UpiPayment implements Payment {  
}
```

Now Spring has:

```
2 Payment Beans  
  |  
  ↓  
Which one?  
  |  
  ↓  
@Primary  
  |  
  ↓  
UpiPayment
```

So the dependency can be resolved without ambiguity.

29. Important Point About @Primary

@Primary means:

Give this Bean priority when multiple candidates are available.

It does **not** mean:

There can only be one Bean.

Both Beans still exist:

```
CardPayment Bean
UpiPayment Bean
```

But UpiPayment gets preference when Spring needs to select a Payment Bean.

30. Example with @Primary

Payment Interface

```
public interface Payment {

    void pay();

}
```

Card Payment

```
@Component
public class CardPayment implements Payment {

    @Override
    public void pay() {
        System.out.println("Paying via Card");
    }

}
```

UPI Payment

```
@Component
@Primary
public class UpiPayment implements Payment {

    @Override
    public void pay() {
        System.out.println("Paying via UPI");
    }
}
```

Order Service

```
@Component
public class OrderService {

    private Payment payment;

    public OrderService(Payment payment) {
        this.payment = payment;
    }

    public void placeOrder() {
        payment.pay();
        System.out.println("Order Placed");
    }
}
```

Spring sees:

```
OrderService
|
↓
Payment required
|
├── CardPayment
|
└── UpiPayment
    ↑
    @Primary
```

Therefore:

```
UpiPayment
  ↓
pay()
  ↓
Paying via UPI
```

The lecture demonstrates this exact idea using Card Payment and UPI Payment.

31. @Qualifier

`@Primary` is useful when we have a **default/preferred Bean**.

But sometimes we don't want a default.

We want to explicitly say:

Inject this particular Bean.

For this, we use:

```
@Qualifier
```

32. Using @Qualifier

Suppose:

```
@Component
public class CardPayment implements Payment {
}
```

and:

```
@Component
public class UpiPayment implements Payment {
}
```

Now both are candidates.

We can explicitly specify:

```
@Component
public class OrderService {

    private Payment payment;

    public OrderService(
        @Qualifier("upiPayment") Payment payment) {

        this.payment = payment;
    }
}
```

Now Spring knows exactly which Bean to inject:

```
OrderService
    |
    ↓
@Qualifier("upiPayment")
    |
    ↓
UpiPayment
```

33. Bean Name with @Component

By default, Spring generally creates the Bean name from the class name with the first letter converted to lowercase.

Example:

```
@Component
public class UpiPayment {
}
```

Bean name:

```
upiPayment
```

Similarly:

```
@Component
public class CardPayment {
}
```

Bean name:

cardPayment

Therefore:

```
@Qualifier("upiPayment")
```

refers to the `UpiPayment` Bean.

34. `@Qualifier` Example

Card Payment

```
@Component
public class CardPayment implements Payment {

    @Override
    public void pay() {
        System.out.println("Paying via Card");
    }
}
```

UPI Payment

```
@Component
public class UpiPayment implements Payment {

    @Override
    public void pay() {
        System.out.println("Paying via UPI");
    }
}
```

Order Service

```
@Component
public class OrderService {

    private Payment payment;

    public OrderService(
        @Qualifier("upiPayment") Payment payment) {

        this.payment = payment;
    }

    public void placeOrder() {
        payment.pay();
        System.out.println("Order Placed");
    }
}
```

Output:

```
Paying via UPI
Order Placed
```

35. @Primary vs @Qualifier

@Primary	@Qualifier
Defines a preferred/default Bean	Explicitly selects a specific Bean
Used on Bean class	Usually used at injection point
Useful when one implementation should be preferred	Useful when a particular implementation is required
Spring chooses it automatically	Developer explicitly tells Spring which Bean to use
Can act as a default	More specific than @Primary

Simple Memory Trick

```
@Primary
  ↓
"Prefer this one"

@Qualifier
  ↓
"Use this exact one"
```

36. What Happens if Both Are Used?

Suppose:

```
@Component
@Primary
public class UpiPayment implements Payment {
}
```

and:

```
@Component
public class CardPayment implements Payment {
}
```

Normally:

```
Payment payment;
```

will receive:

```
UpiPayment
```

But if we explicitly specify:

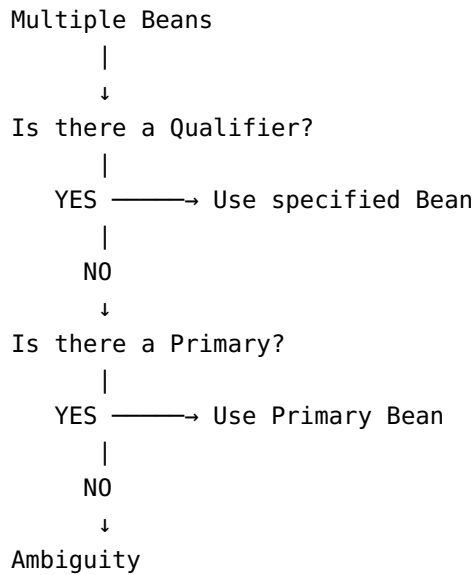
```
@Qualifier("cardPayment")
Payment payment
```

then Spring selects:

CardPayment

because `@Qualifier` gives a more specific instruction.

Conceptually:



37. Complete Dependency Resolution Flow

Suppose we have:

```
Payment
├── CardPayment
└── UpiPayment
```

And:

```
@Component
public class OrderService {

    public OrderService(Payment payment) {
```

```
}  
}
```

Spring needs to resolve:

Payment

It finds two Beans.

Case 1 — No `@Primary`, No `@Qualifier`

```
CardPayment  
UpiPayment  
  ↓  
Ambiguous  
  ↓  
Spring cannot decide
```

Case 2 — `@Primary`

```
CardPayment  
UpiPayment ← @Primary  
  ↓  
UpiPayment selected
```

Case 3 — `@Qualifier`

```
@Qualifier("cardPayment")  
Payment payment
```

Result:

CardPayment selected

38. Why `@Qualifier` Is Useful

Imagine an application has:

```
Payment
├── CardPayment
├── UpiPayment
├── NetBankingPayment
└── WalletPayment
```

Different services may require different payment methods.

For example:

```
OrderService
    ↓
UPI Payment
```

while:

```
SubscriptionService
    ↓
Card Payment
```

Using `@Qualifier`, each service can explicitly choose its required implementation.

39. `@Primary` as a Default Strategy

If your application normally uses UPI:

```
@Component
@Primary
public class UpiPayment implements Payment {
}
```

Then most places can simply request:

```
Payment payment;
```

and Spring will select:

```
UpiPayment
```

Only places that require something different need:

```
@Qualifier("cardPayment")
```

This makes `@Primary` useful for establishing a default implementation.

40. Bean Creation and Dependency Resolution

Spring's overall process can now be understood as:

```
1. Start Application
   ↓
2. Create ApplicationContext
   ↓
3. Read Configuration
   ↓
4. Perform Component Scanning
   ↓
5. Find @Component classes
   ↓
6. Create Bean Definitions
   ↓
7. Create Bean Objects
   ↓
8. Resolve Dependencies
   ↓
9. If multiple candidates:
   |
   | — @Qualifier → Select specified Bean
   |
   | — @Primary → Select preferred Bean
   |
   | — Neither → Ambiguous
   ↓
10. Inject Dependencies
```

↓
11. Beans become ready

41. Bean Definition

Before creating an actual Bean object, Spring maintains information about the Bean.

This information is called:

Bean Definition

A Bean Definition contains metadata/information about how Spring should create and manage a Bean.

Conceptually:

```
Bean Definition
├── Class information
├── Bean name
├── Scope
├── Dependencies
├── Configuration
└── Other metadata
```

Spring keeps Bean Definitions for the Beans it needs to manage.

42. Why Does Spring Need Bean Definitions?

A natural question is:

Why doesn't Spring directly create the objects?

Because Spring needs information about how each Bean should be handled before creating and managing it.

Conceptually:

```
Class
↓
Metadata
```

```
↓  
Bean Definition  
↓  
Bean Creation  
↓  
Dependency Injection  
↓  
Bean Management
```

This allows Spring to manage the complete lifecycle of the Bean.

43. Object vs Bean vs Bean Definition

These three concepts should not be confused.

Object

An ordinary Java object:

```
PaymentService paymentService =  
    new PaymentService();
```

Bean

An object managed by Spring:

```
Spring Container  
↓  
PaymentService Bean
```

Bean Definition

Information that tells Spring how to create/manage that Bean:

```
Bean Definition  
↓  
PaymentService metadata/configuration
```

↓
Spring creates Bean

Remember

Bean Definition $\neq$ Bean

Bean Definition $\rightarrow$ Instructions/metadata

Bean $\rightarrow$ Actual managed object

44. Reflection + Bean Creation

The Java Reflection API becomes important because Spring needs class metadata.

Conceptually:

```
@Component Class
    ↓
Reflection
    ↓
Class Metadata
    ↓
Spring understands class
    ↓
Bean Definition
    ↓
Bean Creation
```

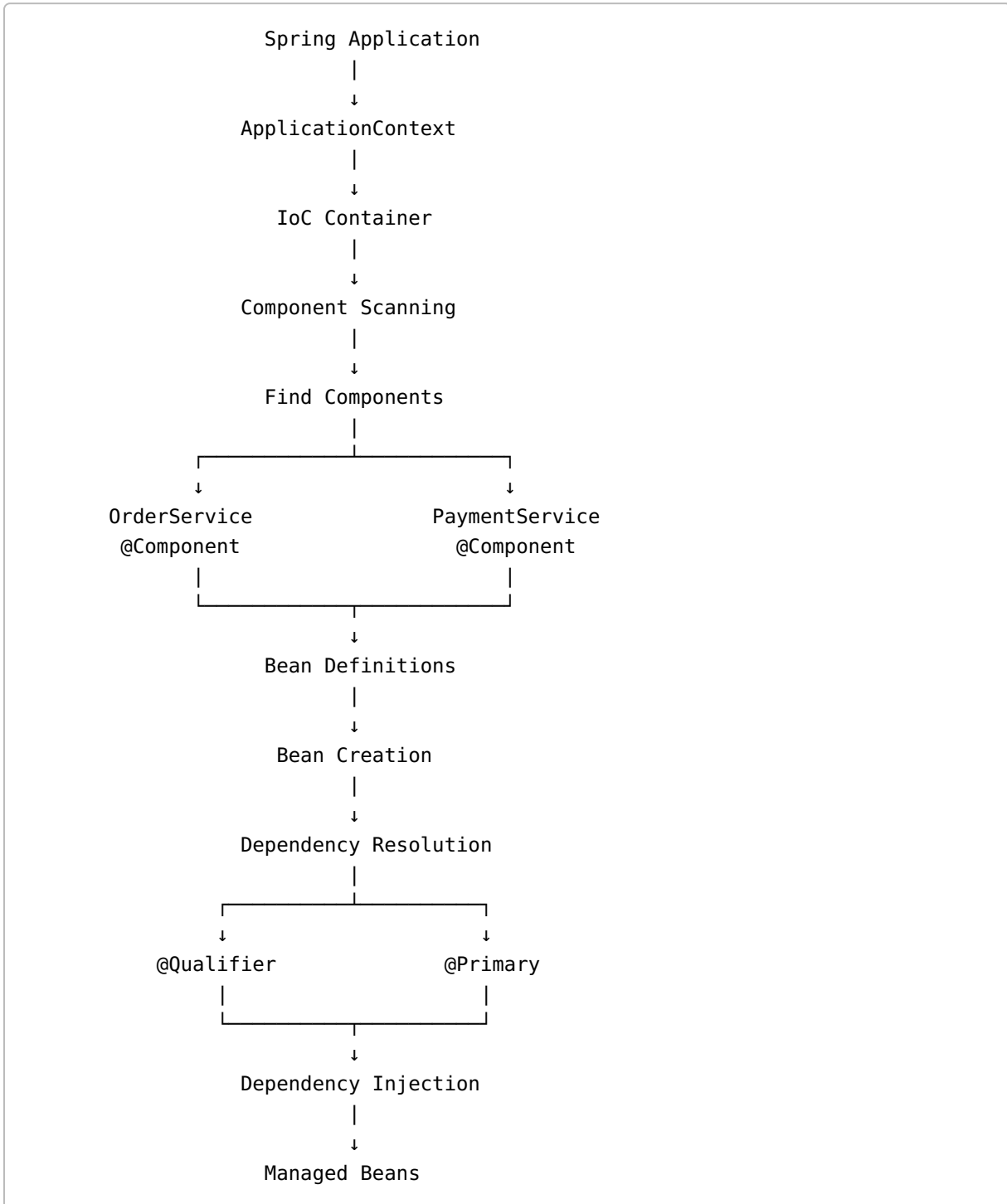
Reflection allows Spring to inspect things such as:

- Class name
- Constructors
- Methods
- Fields
- Annotations
- Other class metadata

This is why the lecture introduces Reflection before explaining Spring's object-management process.

45. Complete Spring Core Architecture

The complete mental model is:



46. Full Example

Payment Interface

```
public interface Payment {  
  
    void pay();  
}
```

CardPayment

```
@Component  
public class CardPayment implements Payment {  
  
    @Override  
    public void pay() {  
        System.out.println("Paying via Card");  
    }  
}
```

UpiPayment

```
@Component  
@Primary  
public class UpiPayment implements Payment {  
  
    @Override  
    public void pay() {  
        System.out.println("Paying via UPI");  
    }  
}
```

OrderService

```
@Component  
public class OrderService {
```

```

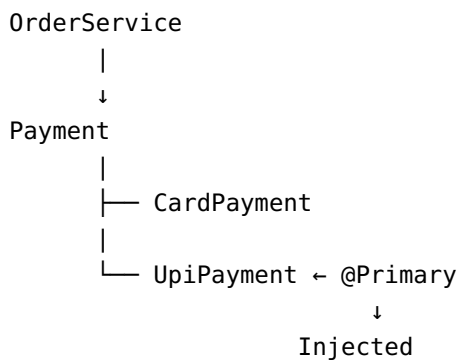
private final Payment payment;

public OrderService(Payment payment) {
    this.payment = payment;
}

public void placeOrder() {
    payment.pay();
    System.out.println("Order Placed");
}
}

```

Because `UpiPayment` is `@Primary`:



47. Using `@Qualifier` Instead

If we want Card Payment specifically:

```

@Component
public class OrderService {

    private final Payment payment;

    public OrderService(
        @Qualifier("cardPayment") Payment payment) {

        this.payment = payment;
    }

    public void placeOrder() {

```

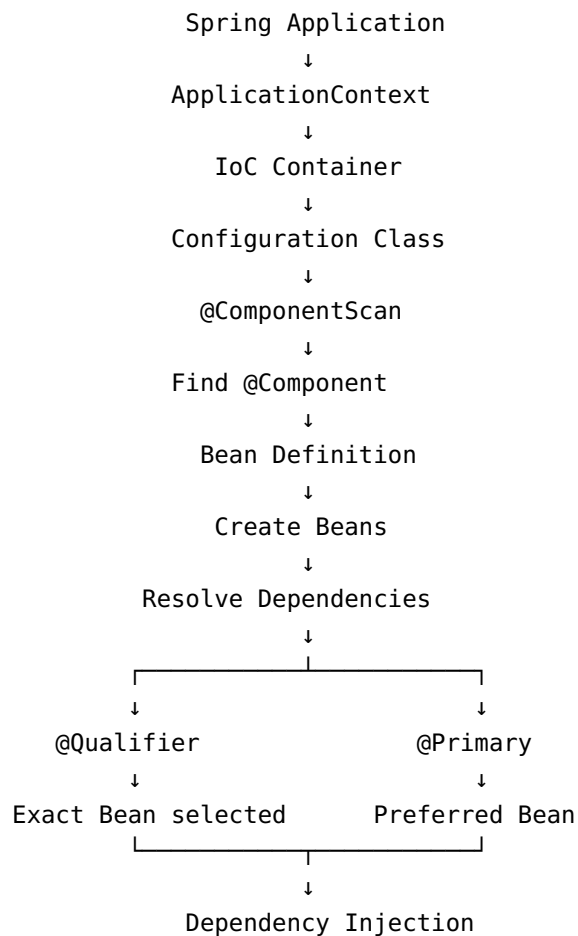
```
        payment.pay();  
    }  
}
```

Now:

```
@Qualifier("cardPayment")  
    ↓  
CardPayment  
    ↓  
Injected into OrderService
```

48. Important Spring Core Flow — Final Revision

Remember this sequence:



↓
Managed Bean Ready

49. Most Important Annotations

Annotation	Purpose
@Component	Marks a class as a Spring-managed component
@Configuration	Marks a configuration class
@ComponentScan	Tells Spring where to find components
@Primary	Gives priority to a Bean when multiple candidates exist
@Qualifier	Selects a specific Bean
@Autowired	Requests dependency injection

50. Final Interview Revision

What is IoC?

Transfer of control of object creation and management from the developer to Spring.

What is an IoC Container?

The Spring container that creates, configures, wires, and manages Beans.

What is a Bean?

An object managed by the Spring IoC Container.

What is `ApplicationContext` ?

A Spring interface representing the application container.

What is `@Component` ?

It marks a class so Spring can discover and manage it as a Bean.

What is `@ComponentScan` ?

It tells Spring where to search for component classes.

What is `@Configuration` ?

It marks a class as a source of Spring configuration.

What is `@Primary` ?

It tells Spring:

```
"Prefer this Bean when multiple candidates exist."
```

What is `@Qualifier` ?

It tells Spring:

```
"Use this specific Bean."
```

Difference between `@Primary` and `@Qualifier`

```
@Primary
  ↓
Default / preferred choice

@Qualifier
  ↓
Explicit / specific choice
```

What is Reflection?

A Java mechanism that allows runtime inspection of class metadata.

What is Bean Definition?

Metadata/information Spring uses to create and manage a Bean.

51. One-Line Memory Map

```
@Component
  ↓
@ComponentScan
  ↓
@Configuration
```

```
↓
ApplicationContext
↓
IoC Container
↓
Bean Definition
↓
Bean Creation
↓
Dependency Resolution
↓
@Qualifier / @Primary
↓
Dependency Injection
↓
Managed Bean
```

52. Final Takeaway

The entire Spring Core concept can be summarized as:

You define your classes and tell Spring which classes it should manage. Spring's IoC Container discovers those classes, creates their Bean Definitions, creates the Beans, resolves their dependencies, injects the required dependencies, and manages the Beans throughout their lifecycle.

When multiple implementations of the same type exist:

```
@Primary
→ choose the preferred/default implementation

@Qualifier
→ choose a specific implementation
```

This is the core foundation on which the higher-level Spring and Spring Boot features are built.